```python
#A-NumPy Operations -1.
import numpy as np
import pandas as pd

df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study
Material\\ecommerce_orders.csv')

# Quantity array
qty_array = df["Quantity"].to_numpy()
print("Quantity array:",qty_array)
# Calculate mean and median
mean_quantity = np.mean(qty_array)
median_quantity = np.median(qty_array)

print("Mean of Quantity is:", mean_quantity)
print("Mean of Quantity is:", median_quantity)
```

```
Quantity array: [3 2 2 1 4 2 4 2 4 3 4 3 3 4 3 1 3 2 4 1 4 2 2 2 3 3 4
4 1 1 4 3 2 4 1 3 4
 4 3 4 3 2 3 3 3 4 4 3 4 1 1 4 4 3 2 4 1 3 4 1 1 2 3 3 2 2 3 3 4 4 2 4
1 4
 4 1 2 1 4 2 4 1 1 4 3 3 1 1 3 3 4 4 3 4 4 2 4 1 4 4]
Mean of Quantity is: 2.8
Mean of Quantity is: 3.0
```

```python
#A-1 answer in different method
array=df["Quantity"]
print(array)
print("Mean:", np.mean(array))
print("Median:", np.median(array))
```

```
0     3
1     2
2     2
3     1
4     4
     ..
95    2
96    4
97    1
98    4
99    4
Name: Quantity, Length: 100, dtype: int64
Mean: 2.8
Median: 3.0
```

```python
#2-
df1=df["TotalAmount"]
print("maximum order price is:", np.max(df1))
print("minimum order price is:", np.min(df1))
```

```
maximum order price is: 220000
minimum order price is: 1500
```

```
#3-
arr1=df["Quantity"]
arr2=df["Price"]
arr3= arr1 * arr2
print("Quantity*Price i.e total revenue is:\n", arr3)
print("The first ten values are:\n" ,arr3[:10])
```

```
Quantity*Price i.e total revenue is:
 0      36000
1       3000
2      40000
3      40000
4      80000
       ...
95     10000
96    220000
97     40000
98    220000
99     20000
Length: 100, dtype: int64
The first ten values are:
 0    36000
1     3000
2    40000
3    40000
4    80000
5    80000
6    48000
7    80000
8     6000
9    36000
dtype: int64
```

```
#4-
quantity = np.array([array])

# Normalization formula: (value - min) / (max - min)
normalized_quantity = (quantity - np.min(quantity)) /
(np.max(quantity) - np.min(quantity))

print("Normalized Quantity:\n", normalized_quantity)
```

```
Normalized Quantity:
 [[0.66666667 0.33333333 0.33333333 0.         1.         0.33333333
  1.         0.33333333 1.         0.66666667 1.         0.66666667
  0.66666667 1.         0.66666667 0.         0.66666667 0.33333333
  1.         0.         1.         0.33333333 0.33333333 0.33333333
```

```
  0.66666667 0.66666667 1.         1.         0.         0.
  1.         0.66666667 0.33333333 1.         0.         0.66666667
  1.         1.         0.66666667 1.         0.66666667 0.33333333
  0.66666667 0.66666667 0.66666667 1.         1.         0.66666667
  1.         0.         0.         1.         1.         0.66666667
  0.33333333 1.         0.         0.66666667 1.         0.
  0.         0.33333333 0.66666667 0.66666667 0.33333333 0.33333333
  0.66666667 0.66666667 1.         1.         0.33333333 1.
  0.         1.         1.         0.         0.33333333 0.
  1.         0.33333333 1.         0.         0.         1.
  0.66666667 0.66666667 0.         0.         0.66666667 0.66666667
  1.         1.         0.66666667 1.         1.         0.33333333
  1.         0.         1.         1.        ]]
```

```python
#B- Pandas Data Analysis- 5-
import pandas as pd
df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study Material\\ecommerce_orders.csv')
print("The five orders are:")
df.head()
```

```
The five orders are:

   OrderID CustomerID    Product  Quantity  Price   OrderDate
Region  \
0      101       C007     Tablet         3  12000  01-04-2024   East

1      102       C020  Headphone         2   1500  02-04-2024   East

2      103       C015     Camera         2  20000  03-04-2024  South

3      104       C011     Tablet         1  40000  04-04-2024  South

4      105       C008     Tablet         4  20000  05-04-2024  South


   PaymentMode  TotalAmount
0   Debit Card        36000
1  Credit Card         3000
2   Debit Card        40000
3          UPI        40000
4          UPI        80000
```

```python
#6-
arr4=df["Product"].unique()
print("Unique products ordered:",arr4)
```

```
Unique products ordered: ['Tablet' 'Headphone' 'Camera' 'Laptop'
'Smartwatch' 'Mobile']
```

```python
#7-
# Group by 'CustomerID' and counting the number of orders
order_count = df.groupby('CustomerID')['OrderID'].count()
order_count.columns = ['CustomerID', 'OrderCount']
print("The average order placed by each customer is:\n", order_count)
```

```
The average order placed by each customer is:
 CustomerID
C001    5
C002    6
C003    5
C004    6
C005    3
C006    2
C007    8
C008    9
C009    6
C010    4
C011    4
C012    8
C013    3
C014    4
C015    8
C016    4
C017    3
C018    5
C019    4
C020    3
Name: OrderID, dtype: int64
```

```python
#8-
avg_order_value = df.groupby('Region')['Price'].mean()
print("The order value by region is:\n", round(avg_order_value))
```

```
The order value by region is:
 Region
East     22333.0
North    24975.0
South    21914.0
West     20611.0
Name: Price, dtype: float64
```

```python
#9-
import pandas as pd

df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study
Material\\ecommerce_orders.csv')

#Convert the 'OrderDate' column to datetime format with dayfirst=True
df['OrderDate'] = pd.to_datetime(df['OrderDate'], dayfirst=True)
```

```
#Defining the date range
start_date = pd.to_datetime('2024-04-02')
end_date = pd.to_datetime('2024-04-05')

#Filtering the DataFrame for orders within the date range
filtered_orders = df[(df['OrderDate'] >= start_date) &
(df['OrderDate'] <= end_date)]

#Display the filtered orders
print(filtered_orders)
```

```
   OrderID CustomerID    Product  Quantity  Price  OrderDate Region  \
1      102       C020  Headphone         2   1500 2024-04-02   East
2      103       C015     Camera         2  20000 2024-04-03  South
3      104       C011     Tablet         1  40000 2024-04-04  South
4      105       C008     Tablet         4  20000 2024-04-05  South

   PaymentMode  TotalAmount
1  Credit Card         3000
2   Debit Card        40000
3          UPI        40000
4          UPI        80000
```

```
#10-
df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study
Material\\ecommerce_orders.csv')
#print(df)
#df = pd.DataFrame(df["TotalAmount"])
# Sorting by 'totalamount' in descending order
sorted_df = df.sort_values(by='TotalAmount', ascending=False )

print("The dataset by TotalAmount in descending order as follows:\
n",sorted_df)
```

```
The dataset by TotalAmount in descending order as follows:
     OrderID CustomerID     Product  Quantity  Price   OrderDate
Region  \
27      128       C003  Smartwatch         4  55000  28-04-2024  North

96      197       C002      Camera         4  55000  06-07-2024  South

90      191       C007      Laptop         4  55000  30-06-2024   East

51      152       C004      Mobile         4  55000  22-05-2024  North

98      199       C016      Laptop         4  55000  08-07-2024  South

..      ...        ...         ...       ...    ...         ...    ...

19      120       C017  Smartwatch         1   1500  20-04-2024  North
```

```
29      130       C019  Smartwatch         1   1500  30-04-2024  North

60      161       C004      Laptop         1   1500  31-05-2024   East

49      150       C018      Camera         1   1500  20-05-2024   East

59      160       C010      Camera         1   1500  30-05-2024  South


    PaymentMode  TotalAmount
27          UPI       220000
96          UPI       220000
90         Cash       220000
51         Cash       220000
98  Credit Card       220000
..          ...          ...
19         Cash         1500
29   Debit Card         1500
60   Debit Card         1500
49         Cash         1500
59  Credit Card         1500

[100 rows x 9 columns]
```

```python
#11-
import pandas as pd
#Total no of orders placed per day.
df['OrderDate'] = pd.to_datetime(df['OrderDate'], dayfirst=True)

# Count number of orders placed per day
orders_per_day = df.groupby(df['OrderDate'].dt.date)
['OrderID'].count().reset_index()

orders_per_day.columns = ['OrderDate', 'TotalOrders']
orders_per_day
```

```
     OrderDate  TotalOrders
0   2024-04-01            1
1   2024-04-02            1
2   2024-04-03            1
3   2024-04-04            1
4   2024-04-05            1
..         ...          ...
95  2024-07-05            1
96  2024-07-06            1
97  2024-07-07            1
98  2024-07-08            1
99  2024-07-09            1

[100 rows x 2 columns]
```

```python
#12-
# Check missing values per column
missing_values = df.isnull().sum().reset_index()
missing_values.columns = ['Column', 'MissingValues']
missing_values
```

```
        Column  MissingValues
0      OrderID              0
1   CustomerID              0
2      Product              0
3     Quantity              0
4        Price              0
5    OrderDate              0
6       Region              0
7  PaymentMode              0
8  TotalAmount              0
```

```python
#13-
# Fill missing values in Price column with the average price
avg_price = df['Price'].mean(skipna=True)
df['Price'].fillna(avg_price, inplace=True)

# Verify if missing values in Price column are handled
missing_after_fill = df['Price'].isnull().sum()
avg_price, missing_after_fill
print(avg_price)
```

```
22275.0

C:\Users\SHREE\AppData\Local\Temp\ipykernel_12416\3239545634.py:4:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Price'].fillna(avg_price, inplace=True)
```

```python
#14-
# Drop rows where Product name is missing
df_cleaned = df.dropna(subset=['Product'])

# Count rows before and after dropping
rows_before = df.shape[0]
rows_after = df_cleaned.shape[0]
```

```
rows_dropped = rows_before - rows_after

rows_before, rows_after, rows_dropped
```

```
(100, 100, 0)
```

```
#15-
import pandas as pd

df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study
Material\\ecommerce_orders.csv')
# Replace missing Quantity values with 1
df['Quantity'] = df['Quantity'].fillna(1)

# Show updated missing values count to confirm
missing_values1 = df.isnull().sum()
print(missing_values1)
```

```
OrderID        0
CustomerID     0
Product        0
Quantity       0
Price          0
OrderDate      0
Region         0
PaymentMode    0
TotalAmount    0
dtype: int64
```

```
#16-
# Remove duplicate rows based on OrderID (keep the first occurrence)
df = df.drop_duplicates(subset="OrderID", keep="first")

# Check number of duplicates remaining
duplicate_count = df.duplicated(subset="OrderID").sum()
print(duplicate_count)
```

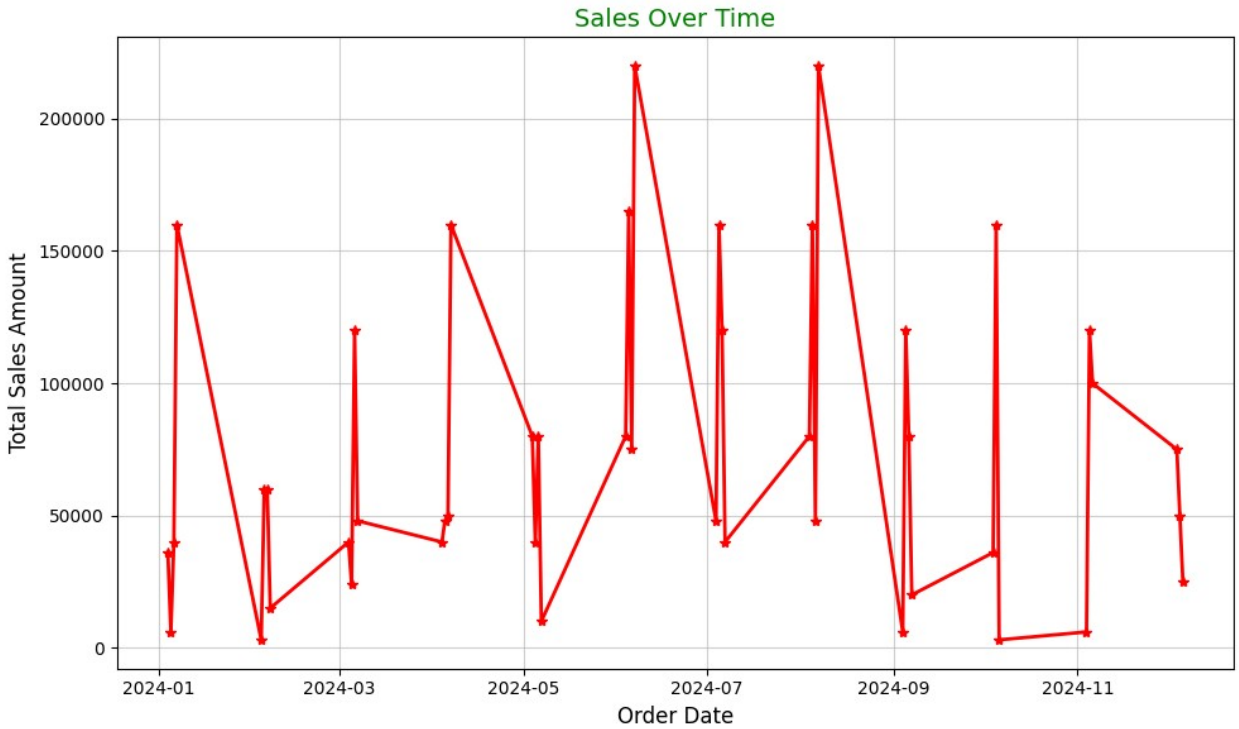
```
0
```

```
#17-
# Check for negative values in Quantity or Price columns
negative_val = df[(df['Quantity'] < 0) | (df['Price'] < 0)]

# Remove rows with negative Quantity or Price
df = df[(df['Quantity'] <= 0) & (df['Price'] <= 0)]

# Count remaining negative values to confirm removal
remaining_negatives = df[(df['Quantity'] < 0) | (df['Price'] <
0)].shape[0]
```

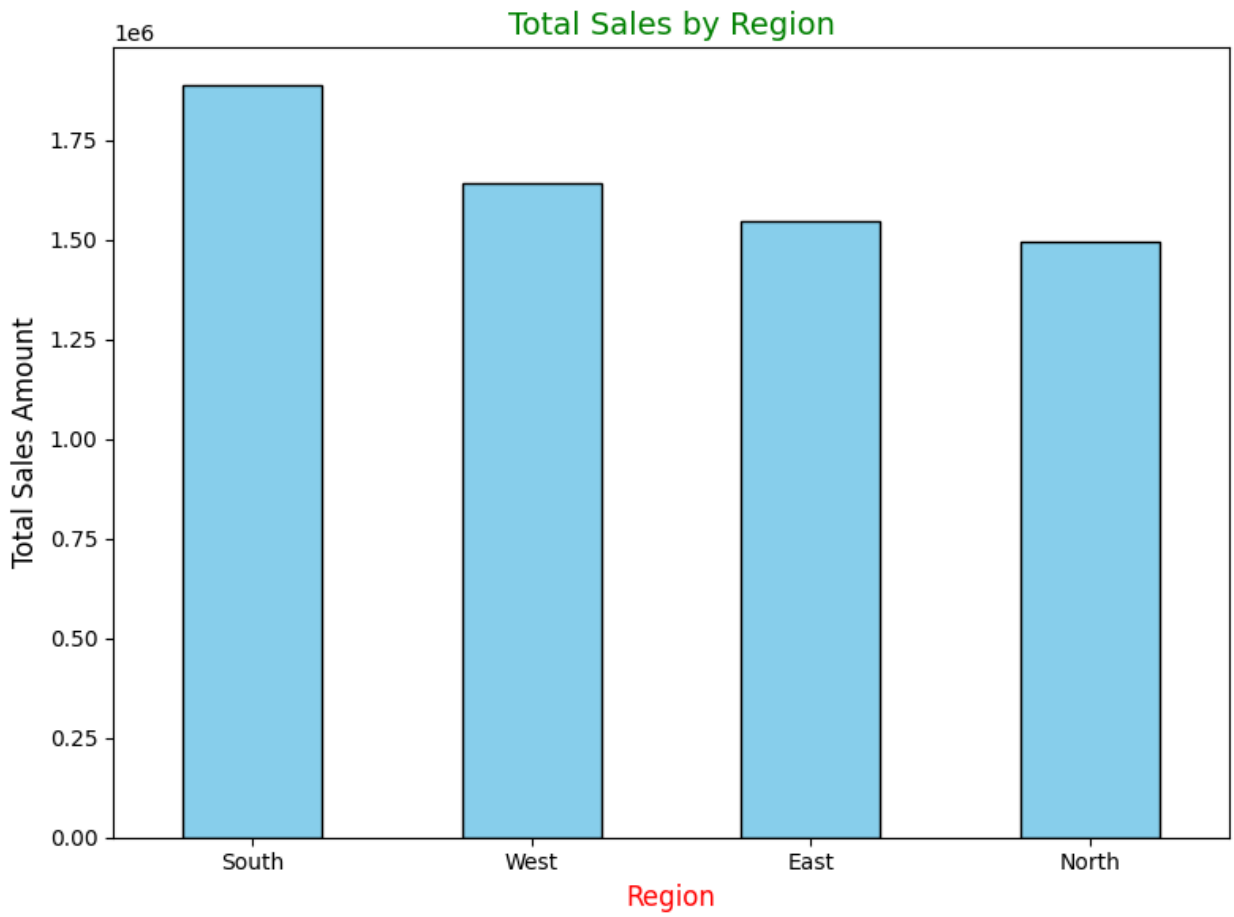
```
print(negative_val)
print(remaining_negatives)
```

```
Empty DataFrame
Columns: [OrderID, CustomerID, Product, Quantity, Price, OrderDate,
Region, PaymentMode, TotalAmount]
Index: []
0
```

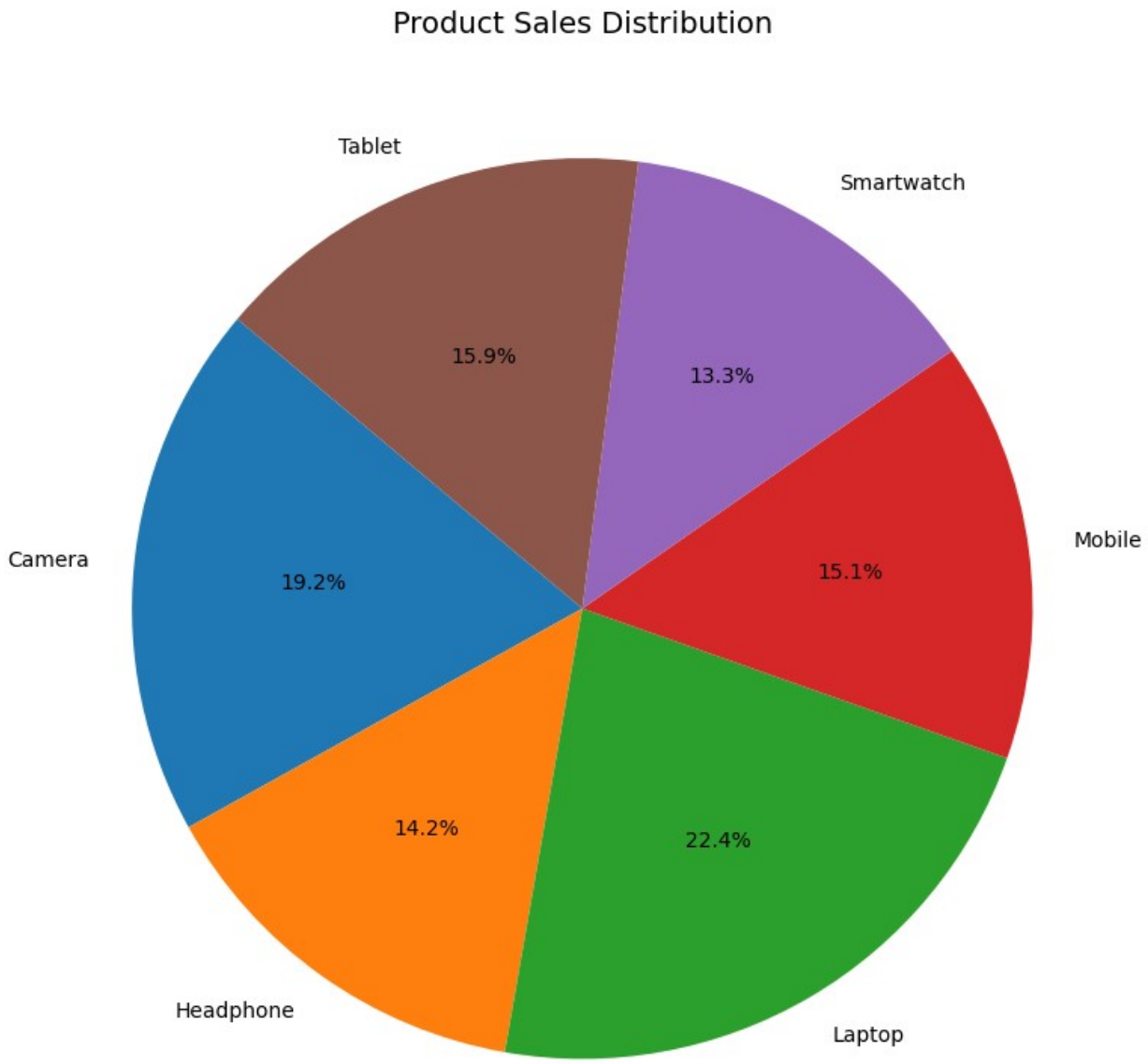
```
#C-Matplotlib/seaborn Visualizations-18-
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study
Material\\ecommerce_orders.csv')
# Convert OrderDate to datetime format
df['OrderDate'] = pd.to_datetime(df['OrderDate'], errors='coerce')

# Group by date and sum TotalAmount
sales_over_time = df.groupby('OrderDate')['TotalAmount'].sum()

# Plot line chart
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time.index, sales_over_time.values, marker='*',
linestyle='-', linewidth=2, color='Red')
plt.title("Sales Over Time", fontsize=14, color='Green')
plt.xlabel("Order Date", fontsize=12)
plt.ylabel("Total Sales Amount", fontsize=12)
plt.grid(True, linestyle="-", alpha=0.6)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```




```
#19-
# Group by Region and sum TotalAmount
sales_by_region = df.groupby('Region')
['TotalAmount'].sum().sort_values(ascending=False)

# Plot bar chart
plt.figure(figsize=(8, 6))
sales_by_region.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title("Total Sales by Region", fontsize=14, color='Green')
plt.xlabel("Region", fontsize=12, color='red')
plt.ylabel("Total Sales Amount", fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

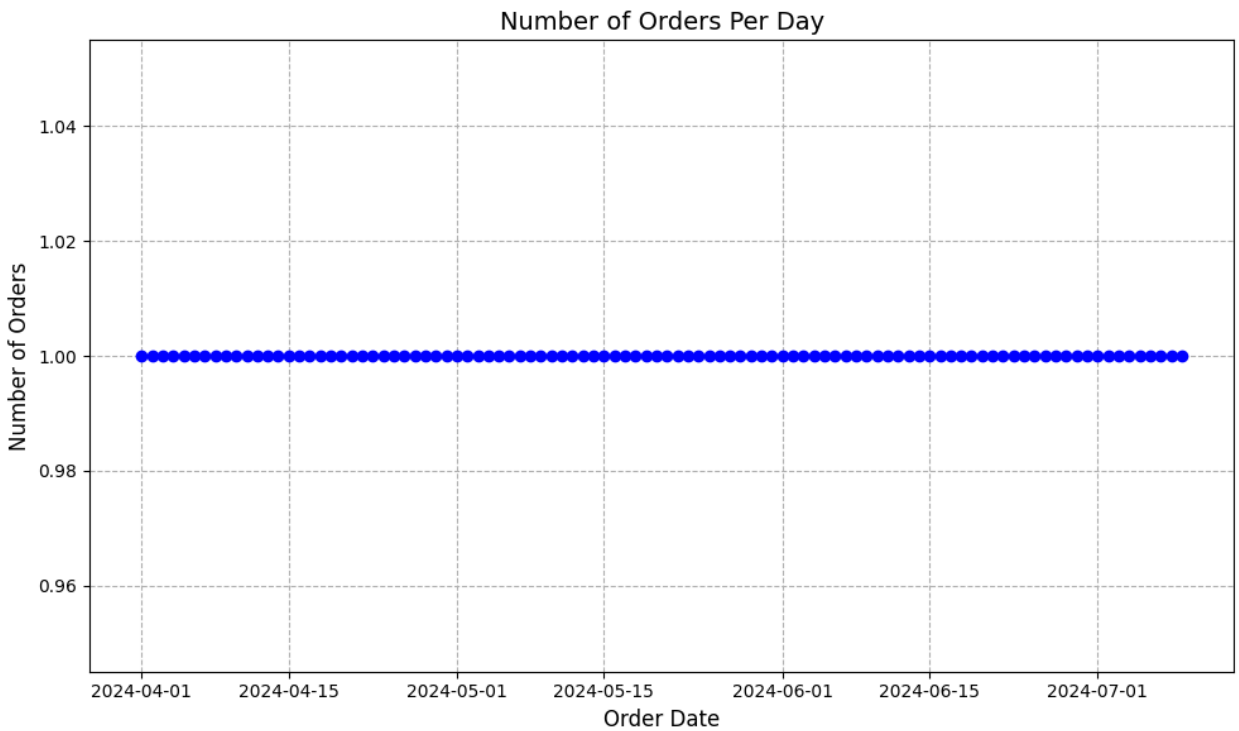


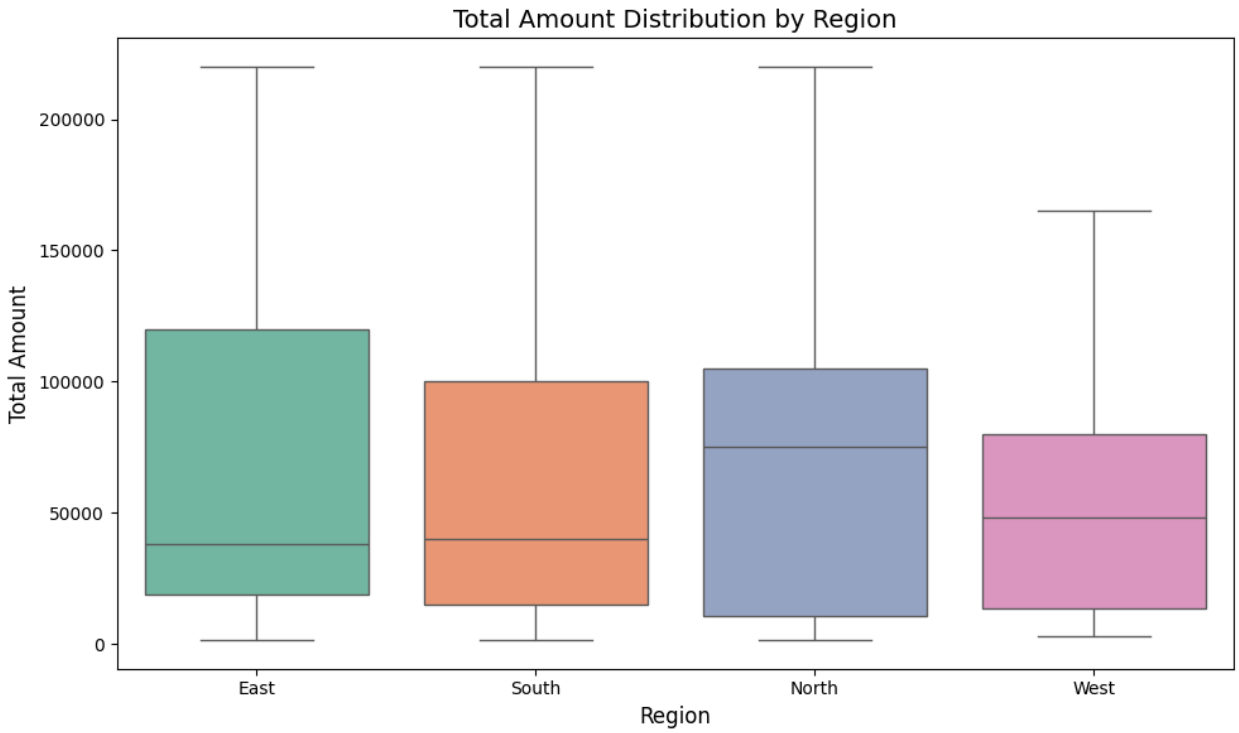

```
#20-
# Group by Product and sum TotalAmount
product_sales = df.groupby('Product')['TotalAmount'].sum()

# Plot pie chart
plt.figure(figsize=(8, 8))
plt.pie(product_sales, labels=product_sales.index, autopct='%1.1f%%',
startangle=140)
plt.title("Product Sales Distribution", fontsize=14)
plt.tight_layout()
plt.show()
```




```
#21-
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# Convert OrderDate to datetime format
df['OrderDate'] = pd.to_datetime(df['OrderDate'], errors='coerce')

# Count number of orders per day
orders_per_day = df.groupby('OrderDate')['OrderID'].nunique()

# Plot line chart
plt.figure(figsize=(10, 6))
```

```
plt.plot(orders_per_day.index, orders_per_day.values, marker='o',
linestyle='-', linewidth=2, color='blue')
plt.title("Number of Orders Per Day", fontsize=14)
plt.xlabel("Order Date", fontsize=12)
plt.ylabel("Number of Orders", fontsize=12)
plt.grid(True, linestyle="--", alpha=1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```




```
#22-
import seaborn as sns

df =  pd.read_csv('C:\\Users\\SHREE\\OneDrive\\Desktop\\Awadhut Study
Material\\ecommerce_orders.csv')

# Create boxplot of TotalAmount by Region
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="Region", y="TotalAmount", palette="Set2")
plt.title("Total Amount Distribution by Region", fontsize=14)
plt.xlabel("Region", fontsize=12)
plt.ylabel("Total Amount", fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
C:\Users\SHREE\AppData\Local\Temp\ipykernel_18412\303151799.py:8:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.boxplot(data=df, x="Region", y="TotalAmount", palette="Set2")
```